RAPPORT TECHNIQUE

LAD Stock — Application de Gestion des Stocks

Génération de l'exécutable (jpackage) & Préparation de la Démonstration

Établissement	Institut National Polytechnique Félix Houphouët-Boigny (INP-HB)
Département	Informatique — Classe IC-GL
Promotion	2025–2026
Date de rédaction	13 avril 2026
Version logiciel	1.0
Logiciel Utilisé	

Réalisé par :

- BROU AKANDA ASSI ARISTIDE
- LENAUD SOKONI EDITH NADIA
- DOUMBIA KARIDJATOU

Table des matières

1. Présentation du projet 3
1.1 Fonctionnalités principales 3
1.2 Environnement de développement 3
2. Architecture technique 4
3. Génération de l'exécutable avec jpackage 5
3.1 Vue d'ensemble de la chaîne de build 5
3.2 Étape 1 — Compilation et création du FAT JAR 5
3.3 Étape 2 — Préparation des ressources jpackage 5
3.4 Étape 3 — Génération de l'installateur avec jpackage 6
3.5 Prérequis pour l'utilisateur final 7
3.6 Script de déploiement complet (Windows) 7
4. Préparation de la démonstration 8
4.1 Préparation de la base de données 8
4.2 Vérifications avant la démonstration 8
5. Scénario de démonstration 9
Phase 1 — Démarrage (1 min) 9
Phase 2 — Tableau de bord (2 min) 9
Phase 3 — Gestion des produits (3 min) 9
Phase 4 — Mouvements de stock (2 min) 9
Phase 5 — Export (2 min) 9
Phase 6 — Gestion des utilisateurs (1 min) 9
Phase 7 — Sécurité (1 min) 9
6. Difficultés rencontrées et solutions 10
7. Recapitulatif 11

1. Présentation du projet

LAD Stock est une application de bureau développée en Java/JavaFX, destinée à la gestion complète des stocks dans un contexte ivoirien. Elle permet à une entreprise de gérer ses produits, ses fournisseurs, ses entrées/sorties de stock et ses utilisateurs, avec un système d'authentification sécurisé et des exports de données.

1.1 Fonctionnalités principales

Module	Description
Tableau de bord	Vue synthétique : total produits, alertes stock, mouvements du jour, valeur totale en FCFA
Produits	Création, modification, suppression, recherche et pagination
Catégories	Gestion des familles de produits
Fournisseurs	Gestion du carnet des fournisseurs
Mouvements	Enregistrement des entrées et sorties de stock avec horodatage
Utilisateurs	Gestion des comptes (rôles ADMIN / GESTIONNAIRE), accessible admin uniquement, affichage du nom de l'utilisateur
Export	Génération de fichiers Excel (.xlsx via Apache POI) et PDF (via iTextPDF)
Sécurité	Hachage des mots de passe, blocage après 3 tentatives échouées (30 secondes)

1.2 Environnement de développement

Élément	Détail
Langage	Java 21 (LTS)
IHM	JavaFX 21
Base de données	MySQL 8.x — base stockmanager_ci
Pilote JDBC	mysql-connector-j 8.3.0
Outil de build	Apache Maven 3.x
Export Excel	Apache POI 5.2.5
Export PDF	iTextPDF 5.5.13.3
IDE utilisé	IntelliJ IDEA

2. Architecture technique

Le projet respecte le patron MVC (Modèle–Vue–Contrôleur). Voici la structure du code source :

```
src/main/
├── java/com/inphb/icgl/stocks/
│   ├── Launcher.java              ← Point d'entrée
│   ├── MainApp.java               ← Application JavaFX principale
│   ├── SplashScreen.java          ← Écran de chargement animé
│   ├── model/                     ← Entités métier (JavaFX Properties)
│   │   ├── Produit.java
│   │   ├── Categorie.java
│   │   ├── Fournisseur.java
│   │   ├── Mouvement.java
│   │   └── Utilisateur.java
│   ├── dao/                       ← Accès base de données (JDBC)
│   │   ├── ProduitDAO.java
│   │   ├── CategorieDAO.java
│   │   ├── FournisseurDAO.java
│   │   ├── MouvementDAO.java
│   │   └── UtilisateurDAO.java
│   ├── repository/                ← Interfaces des DAO
│   ├── controller/                ← Contrôleurs FXML
│   │   ├── LoginController.java
│   │   ├── MainLayoutController.java
│   │   ├── DashboardController.java
│   │   ├── ProduitController.java
│   │   └── ...
│   └── utils/
│       ├── DatabaseConnection.java ← Singleton de connexion MySQL
│       ├── SessionManager.java     ← Gestion de la session utilisateur
│       ├── PasswordUtil.java       ← Hachage des mots de passe
│       ├── AlertUtil.java          ← Boîtes de dialogue
│       └── ExportUtil.java         ← Export Excel et PDF
└── resources/
    ├── fxml/                      ← Vues JavaFX (FXML)
    ├── css/styles.css             ← Feuille de styles (vert pastel)
    └── images/                    ← Logo et fond du Splash Screen
```

Connexion base de données (Singleton) :

```
URL  : jdbc:mysql://localhost:3306/stockmanager_ci
User : root
Pass : (vide — environnement local)
```

3. Génération de l'exécutable avec jpackage

3.1 Vue d'ensemble de la chaîne de build

La génération de l'exécutable se déroule en trois étapes :

```
Code source  →  [mvn package]  →  FAT JAR  →  [jpackage]  →  Installateur .exe / .msi
```

3.2 Étape 1 — Compilation et création du FAT JAR

Le fichier pom.xml est configuré avec le maven-shade-plugin pour produire un JAR autonome contenant toutes les dépendances (JavaFX, MySQL driver, Apache POI, iTextPDF).

Commande :

```
mvn clean package
```

Résultat attendu :

```
target/
└── stockmanager-ci-1.0.jar    ← FAT JAR (toutes dépendances incluses)
```

Le plugin Shade utilise com.inphb.icgl.stocks.Launcher comme classe principale dans le MANIFEST.MF. La classe Launcher est nécessaire car Java 11+ refuse de lancer directement une sous-classe de javafx.application.Application depuis un JAR non modulaire.

Vérification du JAR :

```
java -jar target/stockmanager-ci-1.0.jar
```

3.3 Étape 2 — Préparation des ressources jpackage

Avant d'exécuter jpackage, créer la structure suivante :

```
dist/
├── input/
│   └── stockmanager-ci-1.0.jar    ← copier le FAT JAR ici
└── icon/
    └── lad-stock.ico              ← icône Windows
```

Conversion du logo en .ico (avec ImageMagick) :

```
magick convert src/main/resources/images/logo.png -resize 256x256 dist/icon/lad-stock.ico
```

3.4 Étape 3 — Génération de l'installateur avec jpackage

jpackage est inclus dans le JDK 14+. Avec JDK 21, il est disponible sans configuration supplémentaire.

Commande complète — Windows (.msi) :

```
jpackage \
  --type msi \
  --name "LAD Stock" \
  --app-version "1.0" \
  --description "Application de Gestion des Stocks — INP-HB IC-GL 2025-2026" \
  --vendor "INP-HB IC-GL" \
  --input dist/input \
  --main-jar stockmanager-ci-1.0.jar \
  --main-class com.inphb.icgl.stocks.Launcher \
  --icon dist/icon/lad-stock.ico \
  --dest dist/output \
  --win-menu \
  --win-shortcut \
  --win-dir-chooser \
  --java-options "-Xms128m -Xmx512m"
```

Pour générer un .exe portable (sans installateur) :

```
jpackage \
  --type app-image \
  --name "LAD Stock" \
  --app-version "1.0" \
  --input dist/input \
  --main-jar stockmanager-ci-1.0.jar \
  --main-class com.inphb.icgl.stocks.Launcher \
  --icon dist/icon/lad-stock.ico \
  --dest dist/output \
  --java-options "-Xms128m -Xmx512m"
```

Avantage majeur de jpackage : le JRE est embarqué dans le package final. L'utilisateur final n'a pas besoin d'installer Java sur sa machine.

3.5 Prérequis pour l'utilisateur final

Élément	Requis
Java / JRE	NON (embarqué par jpackage)
MySQL Server 8.x	OUI — doit être démarré
Base stockmanager_ci	OUI — schéma doit être importé
Système d'exploitation	Windows 10 / 11 (64 bits)

3.6 Script de déploiement complet (Windows)

```
@echo off
echo ============================================
echo  LAD Stock — Build et packaging
echo ============================================

REM Etape 1 : Compilation + FAT JAR
call mvn clean package -q
if errorlevel 1 (echo ERREUR : mvn package echoue & pause & exit /b 1)
echo [OK] FAT JAR genere : target\stockmanager-ci-1.0.jar

REM Etape 2 : Preparation du dossier dist
if not exist dist\input mkdir dist\input
if not exist dist\output mkdir dist\output
copy /Y target\stockmanager-ci-1.0.jar dist\input\

REM Etape 3 : jpackage
jpackage --type app-image --name "LAD Stock" --app-version "1.0" ^
  --input dist\input ^
  --main-jar stockmanager-ci-1.0.jar ^
  --main-class com.inphb.icgl.stocks.Launcher ^
  --dest dist\output ^
  --java-options "-Xms128m -Xmx512m"

if errorlevel 1 (echo ERREUR : jpackage echoue & pause & exit /b 1)
echo [OK] Executable genere dans dist\output\
pause
```

4. Préparation de la démonstration

4.1 Préparation de la base de données

a) Démarrer le serveur MySQL

S'assurer que MySQL Server est en cours d'exécution (via les Services Windows ou XAMPP/WAMP).

b) Importer le schéma

```
-- Creer la base
CREATE DATABASE IF NOT EXISTS stockmanager_ci
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE stockmanager_ci;
```

Importer ensuite le script SQL complet du projet (tables utilisateurs, categories, fournisseurs, produits, mouvements).

c) Données de démonstration recommandées

Catégorie	Exemple de produits
Informatique	Ordinateur portable Dell, Imprimante HP
Alimentaire	Riz local 25kg, Huile Palme 5L
Matériaux BTP	Ciment Portland 50kg, Fer à béton 12mm
Bureautique	Ramette de papier A4, Stylos Bille

d) Compte administrateur par défaut

```
Login    : atta
Mot de passe : admin123
Role     : ADMIN
```

Compte de démonstration (rôle GESTIONNAIRE) :

```
Login    : brou
Mot de passe : aristide123
Role     : GESTIONNAIRE
```

4.2 Vérifications avant la démonstration

- MySQL Server démarré et accessible sur le port 3306
- Base stockmanager_ci importée et peuplée avec des données de test
- JAR ou exécutable testé au moins une fois
- Résolution d'écran réglée sur 1280×780 minimum

- Désactiver les notifications Windows pendant la présentation
- Fermer les applications inutiles (antivirus, mises à jour automatiques)
- Préparer un fichier Excel et PDF d'export pour valider la fonctionnalité

5. Scénario de démonstration

Phase 1 — Démarrage (1 min)

- Lancer LAD Stock.exe
- Montrer le Splash Screen animé (fond vert pastel, logo, barre de progression)
- Arriver sur la fenêtre de Connexion
- Se connecter en tant que admin atta / admin123

Phase 2 — Tableau de bord (2 min)

Présenter les 4 cartes de synthèse :

- Nombre total de produits
- Produits en alerte de stock (quantité ≤ stock minimum)
- Mouvements du jour
- Valeur totale du stock en FCFA
- Montrer le tableau des produits en alerte (lignes rouges)

Phase 3 — Gestion des produits (3 min)

- Naviguer vers le module Produits
- Démontrer la recherche dynamique (filtrage en temps réel)
- Ajouter un nouveau produit (référence, désignation, catégorie, prix, stock)
- Modifier un produit existant
- Montrer la pagination si la liste est longue

Phase 4 — Mouvements de stock (2 min)

- Enregistrer une entrée de stock pour un produit donné
- Enregistrer une sortie de stock
- Vérifier la mise à jour de la quantité en stock

Phase 5 — Export (2 min)

- Depuis le module Produits, exporter en Excel (.xlsx)
- Ouvrir le fichier exporté et montrer la mise en forme

- Exporter en PDF et afficher le document

Phase 6 — Gestion des utilisateurs (1 min — admin seulement)

- Créer un nouvel utilisateur avec le rôle GESTIONNAIRE
- Montrer que le menu « Utilisateurs » disparaît si on se connecte avec ce compte

Phase 7 — Sécurité (1 min)

- Se déconnecter
- Tenter 3 connexions avec un mauvais mot de passe
- Montrer le blocage automatique de 30 secondes

6. Difficultés rencontrées et solutions

Difficulté	Cause	Solution retenue
JavaFX runtime components are missing au lancement du JAR	Java 11+ bloque le lancement direct d'une Application depuis un JAR non modulaire	Ajout de la classe Launcher comme intermédiaire dans le MANIFEST.MF
Conflit de modules JavaFX avec le FAT JAR (maven-shade)	Les modules JavaFX natifs (.dll) ne peuvent pas être « shadés »	Utilisation de jpackage qui intègre le JRE complet avec JavaFX
Texte blanc illisible sur fond vert pastel (Splash Screen)	Changement du fond de vert foncé vers vert pastel sans adaptation des couleurs de texte	Mise à jour de toutes les classes CSS et styles inline pour utiliser des couleurs sombres
Blocage de compte sans délai visible	Le blocage était silencieux	Ajout d'un Timeline JavaFX affichant le compte à rebours en temps réel
Encodage des caractères spéciaux en base	Collation MySQL non définie	Ajout de characterEncoding=utf8 dans l'URL JDBC et utf8mb4 sur la base

7. Récapitulatif

LAD Stock v1.0 est une application desktop fonctionnelle et complète, répondant aux besoins de gestion de stocks dans un contexte ivoirien. Développée en Java 21 / JavaFX 21 avec une architecture MVC rigoureuse et une connexion MySQL, elle offre :

- Une interface ergonomique avec thème vert pastel aux couleurs de l'INP-HB
- Des fonctionnalités CRUD complètes sur 5 entités métier
- Un système de sécurité (hachage, blocage, gestion des rôles)
- Des exports professionnels en Excel et PDF
- Un exécutable Windows autonome généré via jpackage (JRE embarqué)

Ce projet constitue une mise en pratique concrète des enseignements reçus en IC-GL (conception orientée objet, bases de données relationnelles, interfaces graphiques, génie logiciel).

Rapport rédigé dans le cadre du projet de fin de module JAVA — INP-HB IC-GL, Promotion 2025-2026